VIVA & PRESENTATION DEFENSE GUIDE

ACADEMIC YEAR 2025-2026 • FINAL EVALUATION

Community Time Bank AI

Complete Technical Walkthrough, Code Logic & Defense Talking Points for Faculty Review

Purpose: Comprehensive Faculty Presentation Reference • **Presenter:** Engineering Project Viva Defense • **Format:** High-Yield Technical Answers & Code References

1. The 30-Second Elevator Pitch (Opening Statement)

> *"Good morning / afternoon Ma'am. My project is **Community Time Bank AI**, a decentralized service reciprocity platform designed for college campuses and local communities. In our application, members exchange real-world assistance—such as Python programming, hardware repair, plant care, and academic tutoring—using **Time Credits** instead of money. The fundamental axiom is: **one verified hour of community assistance equals one base Time Credit (1.0 cr)**, establishing an egalitarian economy. To make this credible and practical, I implemented an **Explainable 100-Point AI Matching Engine**, a **Bilateral Double-Confirmation Settlement Protocol**, and a governed **Fair Credit Multiplier System** (1.0×, 1.25×, 1.5×)."*

2. End-to-End System Architecture

When explaining how the layers interact, describe the clean separation of concerns between client, API, and database:



3. Deep Code Walkthrough (Key Implementation Files)

3.1 The Explainable Matching Engine — `server/src/services/matchingService.ts`

Core Question: *"How does your AI/recommendation algorithm work?"*

What to say: *"Ma'am, instead of relying on an opaque black-box LLM, we engineered a transparent, explainable 100-point multi-factor recommendation engine. It scores every active helper against the request across five weighted parameters:"*

```
MatchScore = S_skill (40%) + S_avail (25%) + S_location (15%) + S_rating (10%) + S_reliability (10%)
```

Factor & Weight	Code Reference	Algorithmic Evaluation Logic
Skill Relevance (40%)	`lines 95-119`	Evaluates proficiency tier: **Expert = 40 pts**, **Intermediate = 32 pts**, **Beginner = 24 pts**. Each additional relevant skill adds +4 pts. Partial category matches yield a 20 pt baseline.
Availability (25%)	`lines 121-141`	Awards **25 pts** for exact weekday overlap with preferred dates. General active schedule slots award 15 pts.
Location Proximity (15%)	`lines 143-155`	Awards **15 pts** for identical neighborhood (e.g., "Downtown") or remote-friendly requests; 7 pts for adjacent city sectors.
Community Rating (10%)	`lines 157-163`	Normalizes the helper's 5-star rating: `(Rating / 5.0) × 10`.
Past Reliability (10%)	`lines 164-172`	Awards **10 pts** for members with ≥ 5 verified completed exchanges, 7 pts for 1–4 exchanges.

3.2 Bilateral Double-Confirmation Settlement — `server/src/services/ledgerService.ts`

Core Question: *"How do you prevent fraud or one person falsely claiming they completed a session?"*

What to say: *"Ma'am, credits never transfer automatically. We enforce a two-party confirmation gate. Only when both parties independently submit their completion confirmation does the atomic transaction execute."*

Code Logic (lines 66–148):

```
const requesterConfirmed = allConfirmations.some(c ⇒ c.userId ═══ booking.requesterId);
const helperConfirmed = allConfirmations.some(c ⇒ c.userId ═══ booking.helperId);

if (requesterConfirmed && helperConfirmed) {
  await prisma.$transaction(async (tx) ⇒ {
    await tx.booking.update({ where: { id: bookingId }, data: { status: 'COMPLETED' } });
    await tx.user.update({ where: { id: booking.requesterId }, data: { creditBalance: { decrement: transferAmount } } })
    await tx.user.update({ where: { id: booking.helperId }, data: { creditBalance: { increment: transferAmount } } });
    await tx.creditTransaction.create({ data: { bookingId, fromUserId: booking.requesterId, toUserId: booking.helperId,
  });
}
```

Why this matters to examiners: `prisma.$transaction` guarantees **atomicity**. If the server loses connection mid-transfer, all operations roll back completely, ensuring zero credit corruption and double-spending protection.

3.3 The Fair Credit Multiplier System — `listings.ts` & `bookings.ts`

Core Question: *"Why should all hours be equal if some tasks require expensive equipment or high physical risk?"*
What to say: *"Ma'am, to accommodate specialized tasks without introducing monetary inflation, we created a governed Fair Credit Multiplier System capped strictly at three tiers:"*

```
TransferAmount = ServiceDurationInHours × AgreedMultiplier
```

- `1.0× STANDARD` Peer tutoring, language conversation, general neighborhood gardening.
- `1.25× INTERMEDIATE` Certified instruction, complex software debugging, specialized tools.
- `1.5× ADVANCED CAP` Component-level hardware repair, hazardous equipment, emergency support. Strictly capped at 1.5×.

Anti-Gouging Enforcement: The client cannot pass arbitrary rates (e.g. 4.0×). The backend strictly validates `ALLOWED_MULTIPLIERS = [1.0, 1.25, 1.5]` and rejects unapproved values with HTTP 400. Once accepted, `agreedMultiplier` is permanently immutable.

3.4 UI Perceived Performance — `client/src/components/SkeletonLoader.tsx`

Core Question: *"What is this SkeletonLoader file, and why is it needed?"*
What to say: *"Ma'am, displaying blank screens or abrupt spinners while awaiting asynchronous REST responses causes Cumulative Layout Shift (CLS) and a jarring user experience. In `SkeletonLoader.tsx`, we created modular skeleton primitives (`CardSkeleton`, `StatCardSkeleton`, `ListItemSkeleton`) that render pulsating placeholder frames matching our cards while data resolves."*

4. Top 5 Viva Questions & Bulletproof Answers

Q1: "Why Time Credits instead of integrating a normal payment gateway like Stripe?"

> *"Ma'am, commercial gig platforms prioritize those with surplus money. In a college campus, many students have valuable skills (like coding or design) but lack disposable cash. Time Banking creates an egalitarian social safety net: one hour of service rendered equals one hour of service received, fostering community solidarity and mutual aid without monetary barriers."*

Q2: "How do you prevent two friends from creating fake bookings to inflate their credits?"

> *"First, our credit economy is closed-loop: to give 2 credits to a helper, the requester must already hold those 2 credits in their wallet balance, which are immediately deducted. Second, every new account starts with only 3 welcome credits, so balance generation is strictly bounded. Third, our Admin Moderation Panel tracks transaction velocity and can freeze suspicious accounts."*

Q3: "Why did you choose Prisma ORM over raw SQL or Mongoose?"

> *"Prisma delivers end-to-end compile-time type safety: schema definitions automatically generate TypeScript types, meaning typos in column names are caught during compilation rather than in production. Furthermore, Prisma interactive transactions (`prisma.$transaction`) simplify complex multi-table ACID operations, guaranteeing financial consistency."*

Q4: "What happens if there is a dispute or a helper does not show up?"

> *"In our Booking state machine, bookings can transition to `DISPUTED`. Clicking 'Report Issue / Dispute' automatically halts the credit transfer, alerts the Campus Administrator at `/admin`, and enables the administrator to inspect communication records and issue a binding credit refund or adjustment with mandatory audit reasons."*

Q5: "Can users cash out their time credits for real fiat currency?"

> *"No Ma'am, direct cash conversions are strictly prohibited to preserve the non-commercial spirit of community service. Instead, surplus credits can be donated to the **Community Credit Pool** to fund hardship grants for peers, or redeemed in the **Perks Catalog** for sponsor-donated cafe coupons or campus event passes."*

5. Step-by-Step Live Demonstration Script

Follow this exact 5-step sequence during your screen-share presentation:

STEP 1: LANDING PAGE & VALUE PROPOSITION (URL: /)

Show the Editorial design, headline manifesto, and scroll to the **Fair Credit Multiplier Calculator**. Demonstrate that moving duration hours automatically calculates fair credit settlement.

STEP 2: PERSONA SWITCHER & SERVICE SEARCH (URL: /find-help)

Click **Alice Johnson** (Requester) on the top persona bar. Search "Python". Show the **Match Card** with the 92% match score and fit highlights. Click **"Request Help"** to reveal the live rate calculation breakdown in the modal.

STEP 3: HELPER ACCEPTANCE & FIRST CONFIRMATION (URL: /bookings)

Switch persona to **Bob Smith** (Helper). Navigate to `/bookings`, accept Alice's request, and click **"Confirm Service Completion"**. Point out the status message: *"Waiting for other member to confirm"* (proving credits have not yet moved).

STEP 4: MUTUAL COMPLETION & ATOMIC LEDGER TRANSFER

Switch back to **Alice**. Click **"Confirm Service Completion"**. Confetti triggers! Navigate immediately to `/wallet` to display the newly generated, immutable transaction record showing base hours, multiplier rate, and new balances.

STEP 5: CAMPUS ADMINISTRATOR AUDIT & GOVERNANCE (URL: /admin)

Switch to **Campus Admin**. Show the **Multiplier Governance** tab, category configuration, dispute mediation queue, and the **Community Credit Pool** hardship grant allocator.

6. Quick Reference Summary Table

Dimension	Engineering Specification & Implementation Details
Frontend Stack	React 18.3.1 • TypeScript 5.6.2 • Vite 6.0.7 • Tailwind CSS 3.4.17 • Lucide React • React Router 6.28.0
Backend Stack	Node.js (ES Modules) • Express 4.21.2 • tsx runtime • Prisma ORM 6.4.1 • SQLite (Dev) / PostgreSQL • JWT • bcryptjs
Database Architecture	15 Relational Models (User, Profile, Category, Skill, UserSkill, Availability, Listing, Booking, Confirmation, CreditTransaction, AdminAuditLog, Review, Report, Redemption, Pool)
Core Algorithms	100-pt Explainable Recommendation Engine (Skill 40%, Avail 25%, Area 15%, Rating 10%, Rel 10%) • Dual-Party Atomic ACID Ledger Settlement
Governance Features	Fair Credit Multipliers (1.0×, 1.25×, 1.5×) • Community Credit Pool (Hardship Grants) • Non-Monetary Perks Catalog

STUDENT CANDIDATE: Chaitanya Toraskar **PROJECT:** Community Time Bank AI **FACULTY EVALUATOR SIGNATURE:** _____